Understanding Lambda function scaling

Concurrency is the number of in-flight requests that your AWS Lambda function is handling at the same time. For each concurrent request, Lambda provisions a separate instance of your execution environment. As your functions receive more requests, Lambda automatically handles scaling the number of execution environments until you reach your account's concurrency limit. By default, Lambda provides your account with a total concurrency limit of 1,000 concurrent executions across all functions in an AWS Region. To support your specific account needs, you can request a quota increase and configure function-level concurrency controls so that your critical functions don't experience throttling.

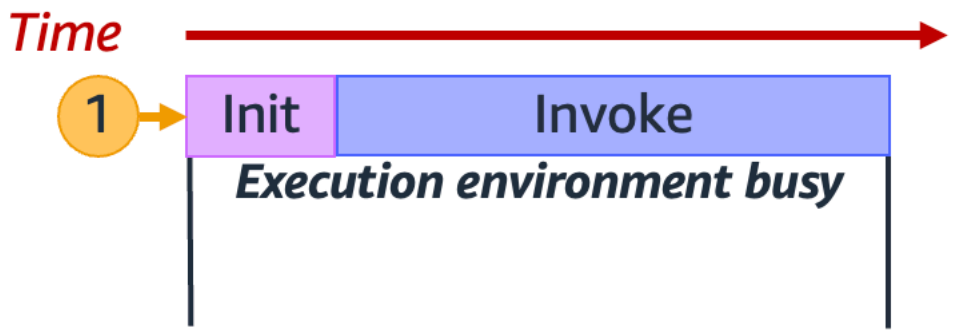
This topic explains concurrency concepts and function scaling in Lambda. By the end of this topic, you'll be able to understand how to calculate concurrency, visualize the two main concurrency control options (reserved and provisioned), estimate appropriate concurrency control settings, and view metrics for further optimization.

Sections

- Understanding and visualizing concurrency
- Calculating concurrency for a function
- Understanding reserved concurrency and provisioned concurrency
- Understanding concurrency and requests per second
- Concurrency quotas
- Configuring reserved concurrency for a function
- Configuring provisioned concurrency for a function
- Lambda scaling behavior
- Monitoring concurrency

Understanding and visualizing concurrency

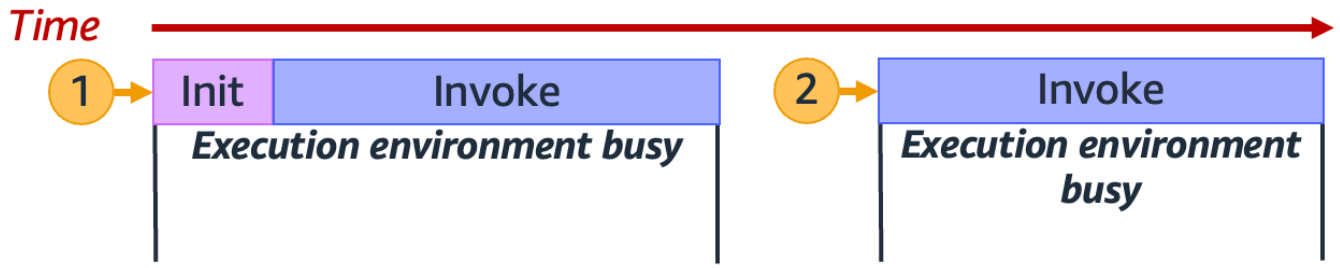
Lambda invokes your function in a secure and isolated execution environment. To handle a request, Lambda must first initialize an execution environment (the Init phase), before using it to invoke your function (the Invoke phase):



> **Note**
>
> Actual Init and Invoke durations can vary depending on many factors, such as the runtime you choose and the Lambda function code. The previous diagram isn't meant to represent the exact proportions of Init and Invoke phase durations.

The previous diagram uses a rectangle to represent a single execution environment. When your function receives its very first request (represented by the yellow circle with label 1), Lambda creates a new execution environment and runs the code outside your main handler during the Init phase. Then, Lambda runs your function's main handler code during the Invoke phase. During this entire process, this execution environment is busy and cannot process other requests.

When Lambda finishes processing the first request, this execution environment can then process additional requests for the same function. For subsequent requests, Lambda doesn't need to re-initialize the environment.

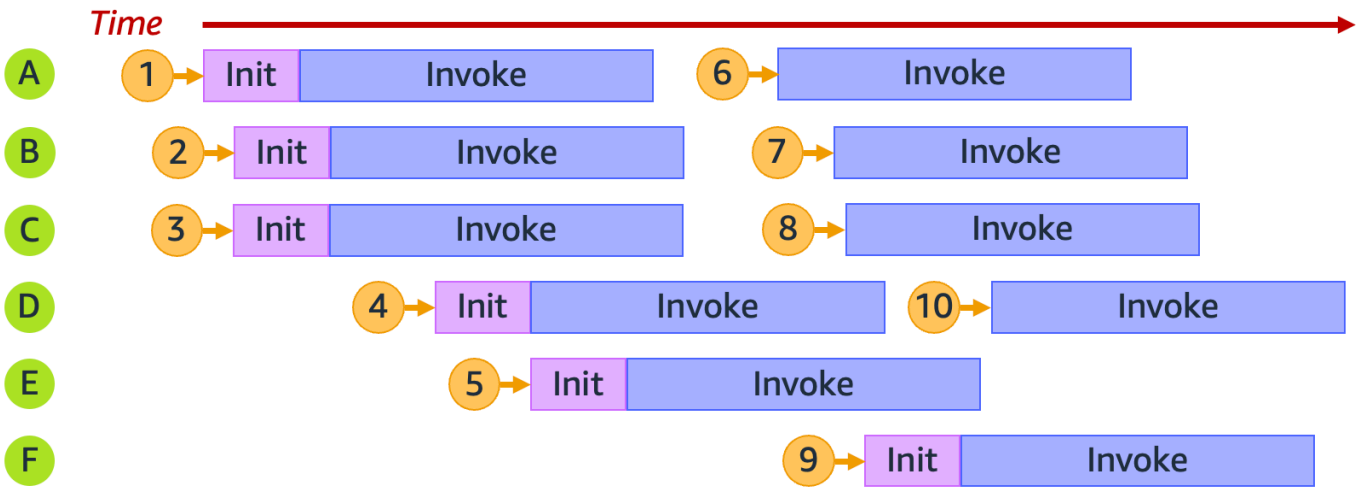


In the previous diagram, Lambda reuses the execution environment to handle the second request (represented by the yellow circle with label 2).

So far, we've focused on just a single instance of your execution environment (that is, a concurrency of 1). In practice, Lambda may need to provision multiple execution environment instances in parallel to handle all incoming requests. When your function receives a new request, one of two things can happen:

- If a pre-initialized execution environment instance is available, Lambda uses it to process the request.
- Otherwise, Lambda creates a new execution environment instance to process the request.

For example, let's explore what happens when your function receives 10 requests:

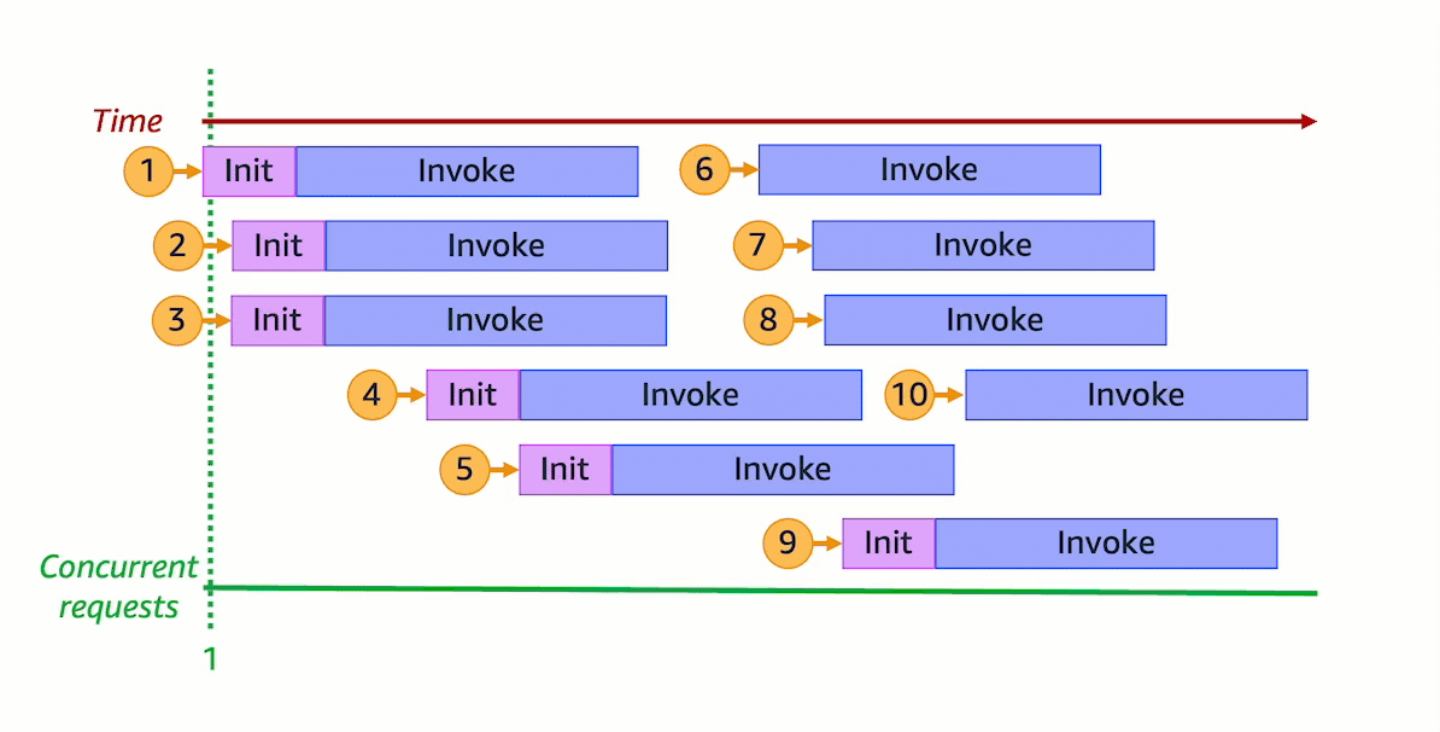
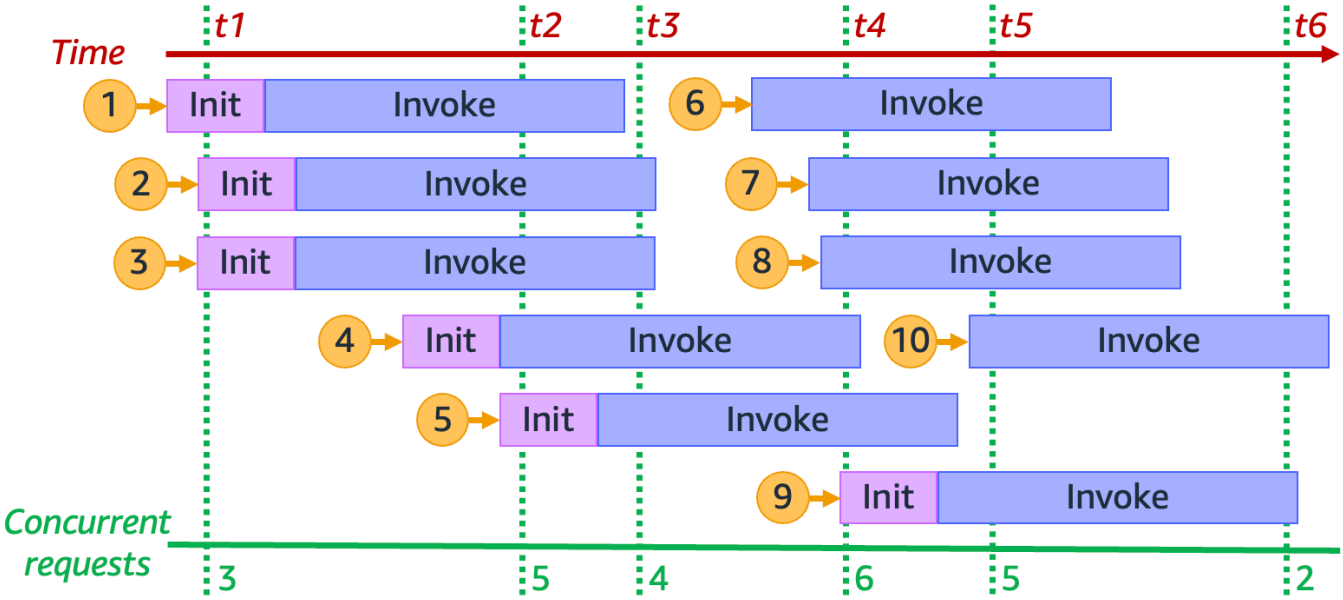


In the previous diagram, each horizontal plane represents a single execution environment instance (labeled from A through F). Here's how Lambda handles each request:

Request	Lambda behavior	Reasoning
1	Provisions new environment **A**	This is the first request; no execution environment instances are available.
2	Provisions new environment **B**	Existing execution environment instance **A** is busy.

Request	Lambda behavior	Reasoning
3	Provisions new environment **C**	Existing execution environment instances **A** and **B** are both busy.
4	Provisions new environment **D**	Existing execution environment instances **A**, **B**, and **C** are all busy.
5	Provisions new environment **E**	Existing execution environment instances **A**, **B**, **C**, and **D** are all busy.
6	Reuses environment **A**	Execution environment instance **A** has finished processing request **1** and is now available.
7	Reuses environment **B**	Execution environment instance **B** has finished processing request **2** and is now available.
8	Reuses environment **C**	Execution environment instance **C** has finished processing request **3** and is now available.
9	Provisions new environment **F**	Existing execution environment instances **A**, **B**, **C**, **D**, and **E** are all busy.
10	Reuses environment **D**	Execution environment instance **D** has finished processing request **4** and is now available.

As your function receives more concurrent requests, Lambda scales up the number of execution environment instances in response. The following animation tracks the number of concurrent requests over time:



By freezing the previous animation at six distinct points in time, we get the following diagram: